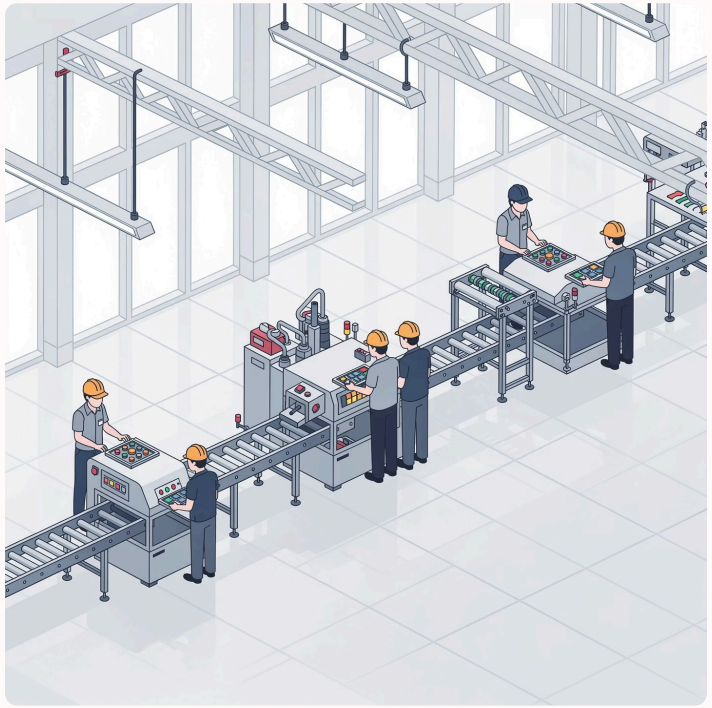
Verificación y Validación con Herramientas

Testing práctico y contratos — Semana 2

UNIDAD 3 · BLOQUE 2



Pruebas Unitarias: Patrón AAA



1

Arrange

Configurar el escenario

2

Act

Ejecutar la lógica

3

Assert

Verificar resultados

Aseveraciones y Casos de Borde

Igualdad y Estado

Correspondencia exacta entre tipos primitivos u objetos complejos

Identidad y Referencia

Verifica si dos variables apuntan al mismo espacio de memoria

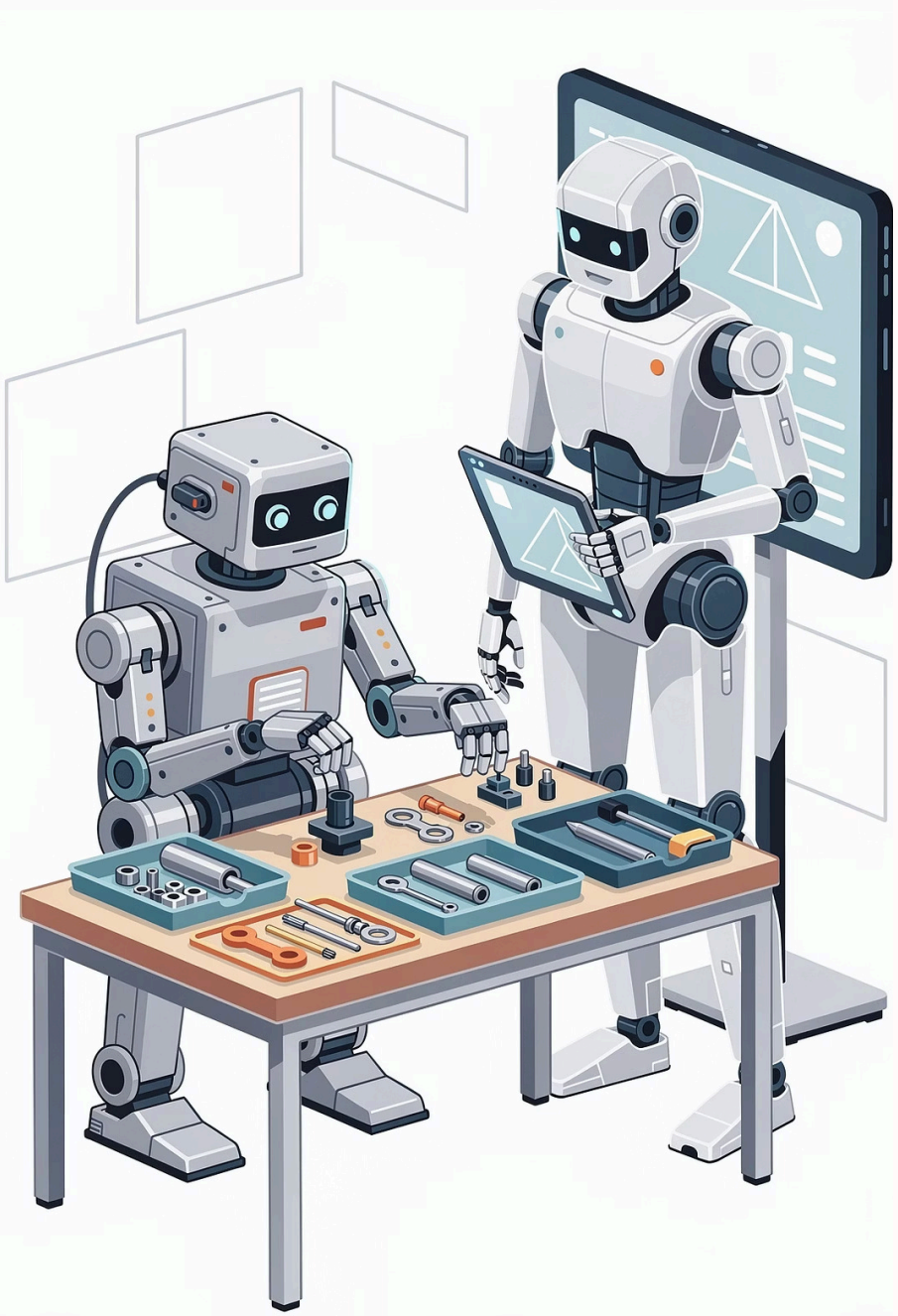
Colecciones

Inspecciona elementos, orden y longitud de arreglos o listas

Inyección de Errores

Fuerza valores nulos, overflow y entradas fuera de rango

El objetivo: capturar excepciones de forma controlada, sin colapsar la aplicación.



Dobles de Prueba: Mocks y Stubs

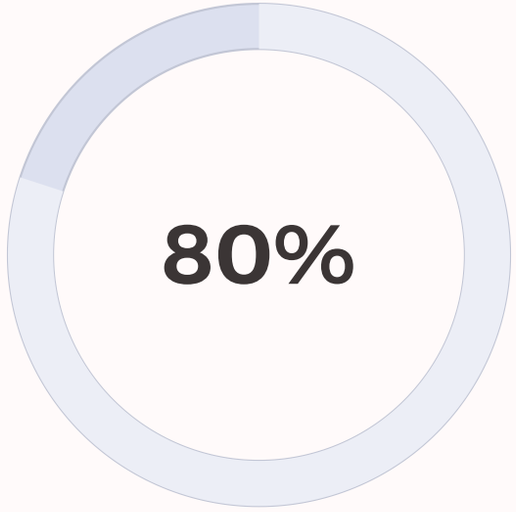
Stub

Objeto estático que devuelve respuestas preconfiguradas. Simula estados de infraestructura sin interactuar con bases de datos o APIs reales.

Mock

Doble dinámico que **audita el comportamiento**: registra qué métodos se invocaron, con qué parámetros, en qué orden y cuántas veces.

Cobertura de Código



Statement Coverage

Estándar industrial mínimo deseable

Statement Coverage

Proporción de líneas ejecutadas al menos una vez. Alta cobertura **no garantiza** ausencia de bugs ante casos de borde.

Branch Coverage

Criterio más riguroso: valida **todos los caminos condicionales** (if/else, switch). Exige al menos un caso verdadero y uno falso por cada bifurcación.

Pruebas de Contrato

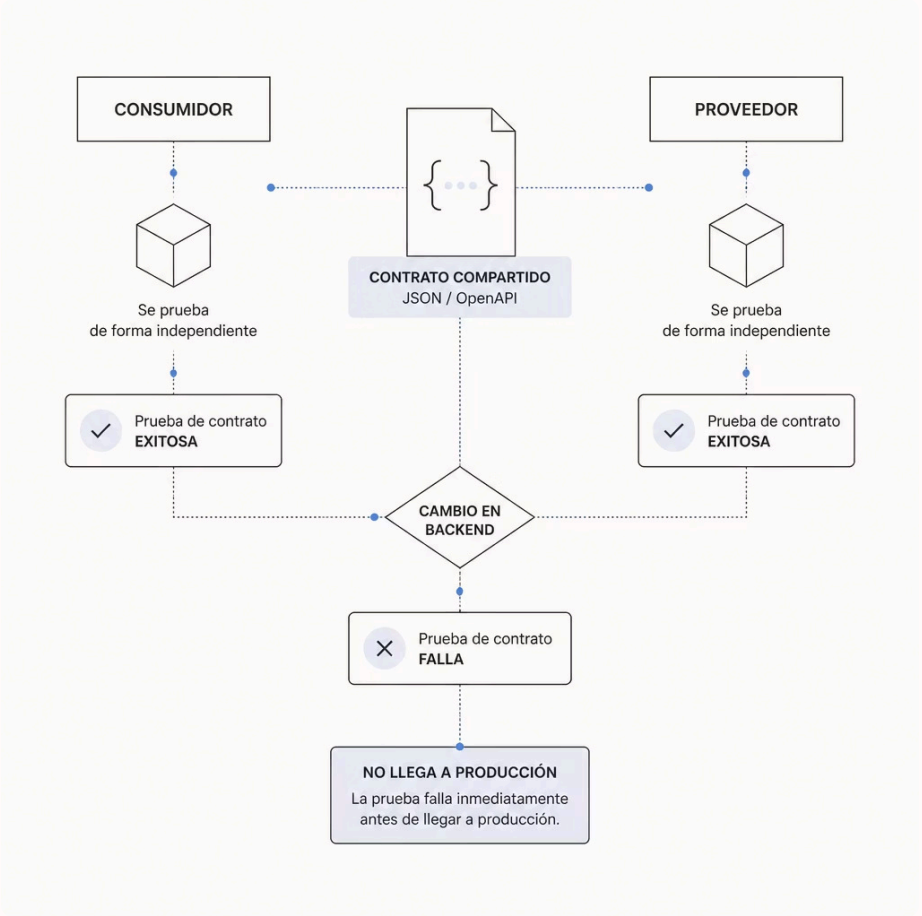
Validan interfaces de comunicación entre servicios desacoplados sin levantar múltiples servidores.

Consumer

Inicia la petición y consume los datos del proveedor

Provider

Expone el endpoint y garantiza respuestas consistentes



Regresión y Smoke Tests

Pruebas de Regresión

Reejecución automática de la suite completa ante cualquier cambio en el repositorio. Red de seguridad contra fallos reintroducidos.

Smoke Test — 3 estados críticos

01

Liveness

Proceso del servidor activo en memoria

02

Readiness

Puertos y sockets responden correctamente

03

Healthcheck

Endpoint base retorna HTTP 200 OK



TDD y BDD Pragmático

TDD — Ciclo Red · Green · Refactor

🔴 Red

Escribe la prueba — debe fallar

🟢 Green

Código mínimo para pasar el test

🔵 Refactor

Limpia y optimiza con seguridad

BDD — Sintaxis Gherkin

Dado (Given)

Contexto y precondiciones del sistema

Cuando (When)

Acción ejecutada por el actor

Entonces (Then)

Consecuencia observable que valida la aseveración